Systems Track

Interview Preparation Series • Professional Edition

Performance Engineering

09

Compiled with LaTeX • June 15, 2026

harsha.somisetty01@gmail.com

In This Edition

1 09 --- Performance Engineering

> **Goal:** Understand how to measure, diagnose, and fix performance problems at every level of the stack. FAANG interviews test whether you think in first principles, not tricks.

1.1 Overview --- What it is

Performance Engineering is the disciplined practice of ensuring a system meets its **speed, resource, and scalability targets** under realistic conditions. It spans:

- **Profiling** — finding where time/resources actually go
- **Bottleneck Analysis** — identifying the single constraint limiting performance
- **Memory management** — leak detection, GC tuning, allocation patterns
- **Latency Optimization** — reducing the time for a single request
- **Throughput Optimization** — increasing requests per unit time
- **Caching Strategies** — trading memory/staleness for speed

The central discipline: **measure first, optimize second.** Performance work without measurement is guesswork.

1.2 Why It Exists

Computers have profound asymmetries in speed:

- A CPU executes an instruction in ~0.3 ns
- A network round-trip to another continent takes ~150,000,000 ns (150 ms)

That is a **500 million× gap.** Software that ignores these asymmetries ends up spending CPU cycles waiting — burning energy and failing users. Performance engineering exists to:

1. **Eliminate waste** — stop waiting for things unnecessarily
2. **Match hardware capabilities** — use CPU pipelines, cache hierarchies, SIMD
3. **Scale economically** — 10× faster = 10× fewer servers = 10× lower cost
4. **Meet SLOs** — users and contracts demand specific latency/availability targets

1.3 Why FAANG Cares (be specific)

Company	Evidence	Lesson
Amazon	Every 100 ms of latency costs ~1% of sales (Vogels, 2006). At Amazon's scale that is hundreds of millions of dollars per year.	Latency is revenue.
Google	”Tail at Scale” (Dean & Barroso, 2013): a single slow backend in a fan-out request makes the whole response slow. At 1,000 backends, p99 of one is p50 of the aggregate.	Tail latency amplifies at scale.
Netflix	Chaos Engineering + performance SLOs. They measure streaming start time to the millisecond — each 500 ms increase noticeably drops subscriber retention.	Real-time systems require p99 discipline.
Facebook/Meta	HipHop VM, Hack type system, HHVM → all performance-motivated. PHP at 1 billion users required compiler-level optimization.	Optimization compounds across requests.
Uber	Switched from PostgreSQL to Cassandra for write throughput; built in-house time-series DB (M3) because OSS could not handle their cardinality.	Bottleneck forces architectural change.

FAANG interview reality: You will be asked ”your service degraded 10×, how do you debug it?” They want: measure → profile → isolate → fix → verify. Never guess. Never optimize without data.

1.4 Core Concepts

Latency vs Throughput vs Bandwidth

These three terms are **not interchangeable** — a classic interview trap.

Term	Definition	Unit	Analogy
Latency	Time for one request to complete end-to-end	ms, µs, ns	Time to drive from A to B
Throughput	Number of requests completed per unit time	req/s, TPS, QPS	Cars crossing a bridge per hour
Bandwidth	Maximum data transfer capacity of a channel	bits/s, MB/s	Width of the highway (lanes × speed limit)

Key insight: **You can have high throughput with high latency** (batch pipeline processing 1 million items but each takes 5 seconds) and **low latency with low throughput** (fast single requests but a system can only handle 10/s).

The relationship via Little's Law (preview):

```
Throughput = Concurrency / Latency
```

To double throughput: either halve latency, or double concurrency (up to the bottleneck).

Interview takeaway: Always clarify which metric matters. Real-time user-facing = latency. Batch = throughput. Both = SLO-driven tradeoff.

Percentiles --- p50, p90, p99, p99.9

Averages lie. A system averaging 10 ms response time might have:

- 90% of requests at 2 ms
- 10% of requests at 82 ms (average = 10 ms)

Those slow 10% are real users having bad experiences.

Percentile definitions:

- **p50 (median):** 50% of requests complete faster than this. Represents the typical user.
- **p90:** 90% complete faster. ”90th percentile” — most users are happy, but 10% are not.
- **p95:** 95th percentile. Common SLO target.
- **p99:** 99% complete faster. 1% of users see this or worse.
- **p99.9 (p999):** 99.9% complete faster. 1 in 1,000 requests.
- **p99.99 (p9999):** 1 in 10,000 requests. Relevant at FAANG scale.

```
Latency
  |
  |      *
  |      **
  |     ****
  |   *******
  | **********   *
  |************  *  *   *
  +------------------------------> Percentile
  p0   p50   p90 p95 p99 p999

 "The tail" = the right side of this distribution
```

Why tail latency matters at scale (Google's insight):

If a request fans out to N backends, the overall latency is the **maximum** of the N individual latencies:

```
P(all N < x) = P(one < x)^N
=> P(max > x) = 1 - (1 - P(one > x))^N
```

With N=100 backends, each at 99th percentile = 1% slow:

```
P(at least one slow) = 1 - (0.99)^100 ≈ 63%
```

63% of fan-out requests will hit a slow backend. At FAANG scale, p99 of one service becomes median (p50) of the system.

Hedged requests (send request to two replicas, use whichever responds first) is Google's solution. Adds ~5% extra load, cuts tail latency dramatically.

Interview takeaway: Never measure only p50. SLOs must be defined on p99 or p99.9 for user-facing systems.

Profiling

Profiling is the act of **measuring where a program spends time or allocates memory** — empirically, not by guessing.

CPU Profiling **Sampling profiler:** Interrupts the program every N milliseconds, records the call stack. Low overhead (~1-5%). Tools: `perf` (Linux), `py-spy` (Python), `async-profiler` (JVM), `pprof` (Go), Instruments (macOS).

Instrumentation profiler: Inserts timing probes at every function entry/exit. Accurate but 2-10× overhead. Tools: gprof, JVM's JFR with method profiling.

What to look for:

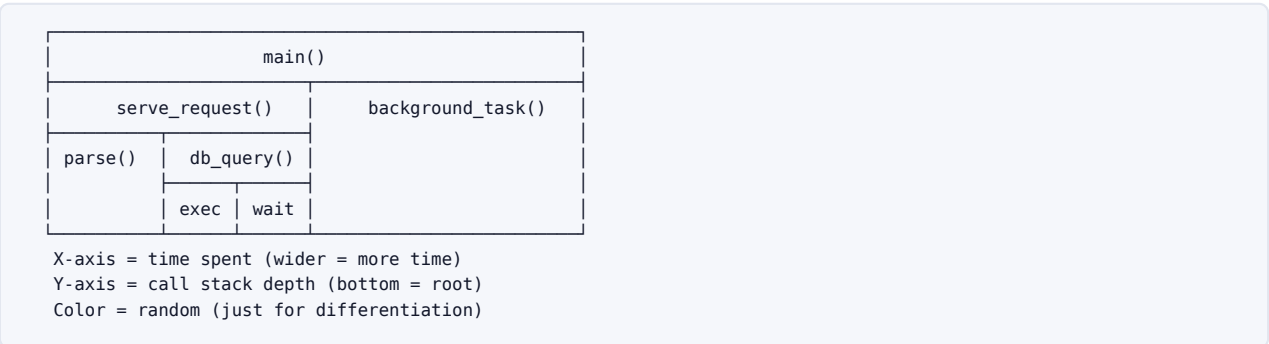
- Functions with highest **self-time** (time spent in the function body itself)
- Functions with highest **cumulative time** (self + all callees)
- **Hot loops** — loops that execute millions of times

Memory Profiling Tracks allocations: which code path allocates how many bytes, and whether they are freed.

Tools: Valgrind/Massif (C/C++), jemalloc heap profiler, JVM heap dumps, `memory_profiler` (Python), Go's `pprof` with `-memprofile`.

Heap dump analysis: Take a snapshot of all live objects. Group by type and trace to GC roots. The shortest path from a GC root to an object is the **retention path** — why it has not been collected.

Flame Graphs Invented by Brendan Gregg. Visualizes sampling profiler output:



How to read: The widest bars at the top are your hottest code. A flat top (plateau) means that function itself is the bottleneck.

Interview takeaway: Flame graphs collapse identical stacks, making hot paths visually obvious. Learn to read them.

Bottleneck Analysis --- CPU vs Memory vs I/O vs Network Bound

Every system has **one binding constraint** at any moment (Theory of Constraints). Finding it before optimizing is critical.

Bound	Symptom	Diagnostic	Fix
CPU-bound	CPU% near 100%, low I/O wait, scales with CPU count	`top`, `perf stat`, flame graphs	Algorithmic optimization, parallelism, SIMD, faster language
Memory-bound	High LLC miss rate, CPU stalls, memory bandwidth saturated	`perf stat -e cache-misses`, `valgrind --cachegrind`	Cache-friendly data structures, reduce working set size, NUMA awareness
I/O-bound	Low CPU%, high `iowait`, disk throughput saturated	`iostat -x`, `iotop`	Async I/O, better indexes, read-ahead, SSD/NVMe upgrade
Network-bound	Low CPU%, NIC saturated, high packet drop	`netstat -s`, `ss`, `sar -n DEV`	Compression, batching, CDN, larger MTU, more NICs
Lock-bound (Concurrency)	Many threads, low CPU utilization, high context switches	`perf lock`, `jstack` thread dumps	Reduce lock scope, lock-free structures, sharding

Diagnostic checklist for a degraded service:

1. Check CPU% — if >80%, suspect CPU-bound
2. Check memory usage & swap — if swap active, memory-bound
3. Check I/O wait (`%wa` in top) — if >10%, suspect I/O
4. Check network saturation (`sar -n DEV`)
5. Check thread count and lock contention

Memory Leaks

A **memory leak** is a bug where allocated memory is never freed (in GC languages: references are held longer than needed, preventing collection).

In manual memory languages (C/C++):

- Allocated with `malloc`/`new`, never `free`/`deleted`
- Tools: Valgrind (detects leaks at program exit), AddressSanitizer (fast, compile-time instrumentation)

In GC languages (Java, Python, Go):

- Memory is ”leaked” by keeping references alive inadvertently
- Common causes:
  - Static collections that grow without bound (`static List<Event> events`)
  - Listeners/callbacks registered but never deregistered
  - Caches without eviction policies (unbounded `HashMap`)
  - Thread-local variables not cleaned up
  - Inner class holding reference to outer class

Detection pattern (Java):

1. Enable GC logging: `-XX:+PrintGCDetails`
2. Take heap dump: `jmap -dump:live,format=b,file=heap.hprof <pid>`
3. Analyze with Eclipse MAT or VisualVM: look for ”Leak Suspects” report
4. Find the **dominator** — the object retaining the most heap

Detection pattern (Python):

```source
import tracemalloc
tracemalloc.start()
# ... code under test ...
snapshot = tracemalloc.take_snapshot()
top_stats = snapshot.statistics('lineno')
for stat in top_stats[:10]:
    print(stat)
```

Interview takeaway: In GC languages, "leak" means a reference is kept alive. The GC cannot collect what is still reachable. Always ask: what is the retention path?

Caching Strategies

Caching trades **memory (and staleness risk) for latency**. Understanding cache placement and eviction is critical.

Cache Placement Levels

```
Request → [CDN Cache] → [Load Balancer] → [App Server]
                                              ↓
                                    [In-Process Cache (e.g., Guava)]
                                              ↓
                                    [Distributed Cache (Redis)]
                                              ↓
                                         [Database]
```

Each layer reduces load on the next. Cache hit at CDN = zero application server load.

Cache Eviction Policies

Policy	Description	Best For
LRU (Least Recently Used)	Evict the item not accessed for the longest time	General purpose; temporal locality
LFU (Least Frequently Used)	Evict the least-accessed item	Repeated access patterns
FIFO	Evict oldest item regardless of access	Simple; streaming data
ARC (Adaptive Replacement Cache)	Hybrid: tracks both recency and frequency	Production systems (ZFS, PostgreSQL)
TTL-based	Evict after a time-to-live expires	Cache invalidation via expiry

Cache Consistency Problems

- **Cache stampede (thundering herd):** Cache entry expires; 1000 threads all miss and all go to DB simultaneously. Fix: probabilistic early expiration, request coalescing, mutex on first load.
- **Cache coherence:** Multiple cache nodes hold stale copies. Fix: TTL + active invalidation (pub/sub on writes).
- **Cold start:** Cache is empty after restart. Fix: pre-warming from DB, lazy-loading with graceful degradation.

Read vs Write Strategies

Strategy	Write Path	Read Path	Consistency	Use Case
Cache-aside (Lazy)	Write DB directly; invalidate cache	Read cache; on miss, load from DB	Eventual	Most common; flexible

Strategy	Write Path	Read Path	Consistency	Use Case
Write-through	Write to cache AND DB synchronously	Read cache (always fresh)	Strong	Read-heavy, low write tolerance
Write-behind (Write-back)	Write to cache; async flush to DB	Read cache	Eventual (risk of loss)	Write-heavy, can tolerate loss
Read-through	Write to DB	Cache fetches from DB on miss automatically	Eventual	Simpler client code

Interview takeaway: Cache-aside is default. Write-through for strong consistency. Write-behind for write-heavy workloads. Always consider what happens on cache failure.

GC Tuning

Garbage collectors stop-the-world (STW) or run concurrently to reclaim memory. Poor GC tuning is a **major source of latency spikes** in Java services.

Java GC generations:

```
Young Gen (Eden + Survivors)  |  Old Gen (Tenured)  | Metaspace
     Minor GC (fast, <10ms)   |   Major/Full GC (slow, can be >1s)
```

Key JVM flags:

- `-Xms` / `-Xmx`: initial and max heap (set equal to avoid resizing pauses)
- `-XX:+UseG1GC`: G1 (Garbage First) — default in Java 9+; good for large heaps
- `-XX:+UseZGC`: ZGC — pause times < 1 ms, designed for p99 latency (Java 15+)
- `-XX:MaxGCPauseMillis=200`: target pause time hint to G1
- `-Xss`: thread stack size (reduce if many threads)

Signs of GC pressure:

- Frequent minor GCs (every few seconds)
- Old gen growing without bound → eventually Full GC → multi-second pause
- High CPU% but actual work is low (GC consuming CPU)

Fix strategies:

1. Increase heap (`-Xmx`) if it is too small
2. Reduce object allocation rate (object pooling, avoid boxing primitives)
3. Move to ZGC/Shenandoah for latency-sensitive applications
4. Profile allocations with async-profiler allocation profiling mode

Latency Optimization Techniques

Batching: Group multiple small operations into one larger operation. Amortizes per-operation overhead (syscall cost, network RTT, DB commit overhead).

Pipelining: Send multiple requests without waiting for prior responses. Reduces head-of-line blocking. Redis pipelines, HTTP/2 multiplexing.

Async I/O: Do not block a thread while waiting for I/O. Use callbacks, futures, or async/await. Allows one thread to service thousands of concurrent I/O operations (Node.js model, Java NIO, Python asyncio).

Connection pooling: Establishing TCP+TLS connections is expensive (~100 ms+). Pool connections and reuse them. Without a pool, each DB query incurs connection overhead.

Read replicas: Route read traffic to replicas, reducing load on primary. Scales read throughput linearly with replica count (eventual consistency).

Compression: Trade CPU for bandwidth. Gzip/Brotli for HTTP. LZ4/Zstd for internal RPC (faster than gzip, slightly less compression ratio).

Protocol selection: REST (HTTP/1.1) < gRPC (HTTP/2 + Protobuf) < raw TCP with custom binary protocol in terms of overhead.

Throughput Optimization Techniques

Horizontal scaling: Add more instances. Effective only if the bottleneck is stateless compute.

Vertical scaling: Larger machine. Simpler but limited by single-machine capacity and cost per unit.

Load balancing algorithms:

- Round-robin: Simple, ignores load
- Least connections: Routes to least busy server
- Consistent hashing: Routes same key to same server (useful for caching)
- Power of two choices: Pick two random servers, route to the less loaded

Work queue / task queue: Decouple producers from consumers. Producers enqueue work (Kafka, RabbitMQ, SQS); consumers process at their own rate. Enables backpressure.

Sharding: Partition data horizontally across nodes. Each shard handles a subset. Scales write throughput. Trade-off: cross-shard queries are expensive.

1.5 Key Laws & Methods

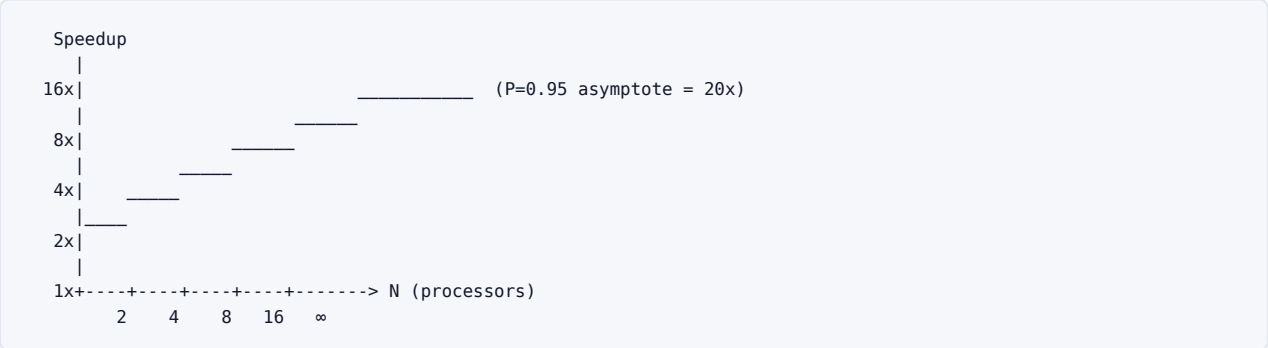
Amdahl's Law

Defines the **theoretical maximum speedup** from parallelizing a program.

```
S(N) = 1 / [(1 - P) + P/N]

Where:
  S(N) = speedup with N processors
  P    = fraction of program that is parallelizable
  N    = number of processors
```

Intuition:



Critical insight: If 5% of your code is serial (P=0.95), maximum speedup is **20×, no matter how many CPUs you add.**

Corollary (Gustafson's Law): Amdahl's Law assumes fixed problem size. If you scale problem size with processors, efficiency is much better. Real HPC workloads do this.

Interview takeaway: Amdahl's Law tells you to eliminate serial bottlenecks before throwing more hardware at a problem.

Little's Law

From queuing theory. Relates three fundamental quantities:

```
L = λ × W

Where:
```

```
L = average number of items in the system (concurrency)
λ = average arrival rate (throughput)
W = average time an item spends in the system (latency)
```

Rearranged:

```
Throughput (λ) = Concurrency (L) / Latency (W)
```

Practical meaning:
If your service has latency W=100ms and you have L=10 threads in flight:

- Throughput = 10 / 0.1s = **100 req/s**

If latency doubles to 200ms but concurrency stays at 10:

- Throughput = 10 / 0.2s = **50 req/s** — throughput halves!

To maintain throughput when latency increases: you must increase concurrency proportionally (more threads, more connections, more workers).

Common interview scenario: "Our throughput dropped by 50%, why?" Little's Law tells you: either latency doubled, or concurrency halved (e.g., thread pool exhaustion, connection pool exhaustion).

Interview takeaway: Little's Law is the master equation of capacity planning. If latency goes up and you do not add concurrency, throughput falls.

USE Method (Brendan Gregg)

For every **resource** in the system, check three things:

```
USE = Utilization + Saturation + Errors

Resource examples: CPU, memory, disk, network, mutex
```

Metric	Definition	Example
Utilization	% of time the resource is busy (or % capacity used)	CPU at 80%
Saturation	Degree to which the resource has extra work queued (cannot service immediately)	Run queue length > 0, disk I/O queue depth
Errors	Error events (both correctable and not)	Network packet drops, disk errors

Workflow:

1. List all resources (CPU per core, DRAM, each NIC, each disk, each interconnect)
2. For each resource, measure U, S, E
3. High U + any S = that resource is the bottleneck

When to use USE: Infrastructure monitoring, capacity planning, diagnosing a slow system at the hardware/OS level.

```
USE Checklist (abbreviated):
  CPU:     util=mpstat, sat=vmstat r-col, errors=dmesg MCE
  Memory:  util=free, sat=vmstat si/so (swap), errors=dmesg ECC
  Network: util=sar -n DEV (%util), sat=netstat overruns, errors=netstat -s
  Disk:    util=iostat %util, sat=iostat await, errors=smartctl
```

RED Method (Tom Wilkie / Weaveworks)

For every **service/microservice**, check:

```
RED = Rate + Errors + Duration

Rate     = requests per second (throughput)
Errors   = number/rate of failed requests
Duration = distribution of request latencies (p50, p99)
```

When to use RED: Application-level monitoring, microservices dashboards, SLO definition.

Method	Scope	Focus
USE	Infrastructure resources	Hardware/OS bottlenecks
RED	Services / microservices	User-facing request health

The four golden signals (Google SRE book): Latency, Traffic, Errors, Saturation — USE and RED together cover all four.

Interview takeaway: USE tells you where hardware is constrained. RED tells you what users experience. Use both at different layers.

The Memory Hierarchy (Latency Numbers Every Engineer Should Know)

```
Memory Hierarchy — Latency & Capacity (approximate, 2024):

Level              Latency      Bandwidth    Size
──────────────────────────────────────────────────
L1 Cache           ~0.5 ns      ~1 TB/s      32-64 KB
L2 Cache           ~5 ns        ~400 GB/s    256 KB - 1 MB
L3 Cache (LLC)     ~20-40 ns    ~200 GB/s    8-64 MB
Main RAM (DRAM)    ~100 ns      ~50 GB/s     8-512 GB
NVMe SSD           ~100 µs      ~7 GB/s      1-8 TB
SATA SSD           ~500 µs      ~550 MB/s    1-4 TB
HDD                ~5-10 ms     ~150 MB/s    1-20 TB
Network (LAN)      ~100 µs RTT  ~12.5 GB/s   —
Network (WAN/US)   ~10-50 ms    ~100 MB/s    —
Network (global)   ~150 ms      variable     —

Scale visualization:
L1    |  (1 ns)
L2    |||||  (5 ns)
L3    ||||||||||||||||||||  (20 ns)
RAM   ||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||  (100 ns)
NVMe  [100,000 ns = 100 µs — 100,000x slower than L1]
SSD   [500,000 ns = 500 µs]
HDD   [10,000,000 ns = 10 ms]
```

Cache locality — the most impactful CPU optimization after algorithmic complexity:

- **Temporal locality:** Use the same data repeatedly → it stays in cache (loop over an array, reuse variables)
- **Spatial locality:** Access data in sequential memory order → cache lines prefetched automatically (row-major traversal of 2D array vs column-major)

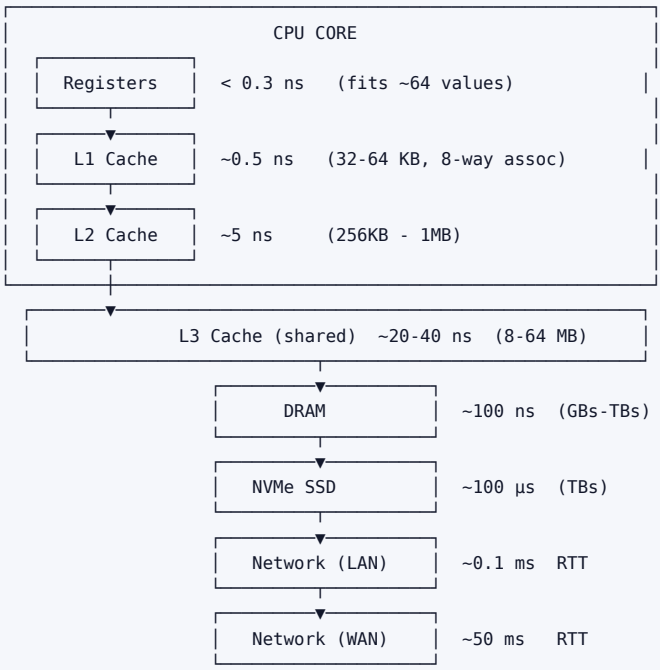
Cache line: Modern CPUs load 64 bytes at a time. If you access `array[0]`, `array[1]`...`array[7]` (8 × 8-byte doubles) all arrive in one cache miss. Accessing `array[0]`, `array[8]`, `array[16]`... causes a cache miss every access.

False sharing: Two threads modify different variables that happen to be on the **same cache line**. The cache coherency protocol bounces the cache line between cores, creating invisible contention. Fix: pad structs to 64-byte alignment.

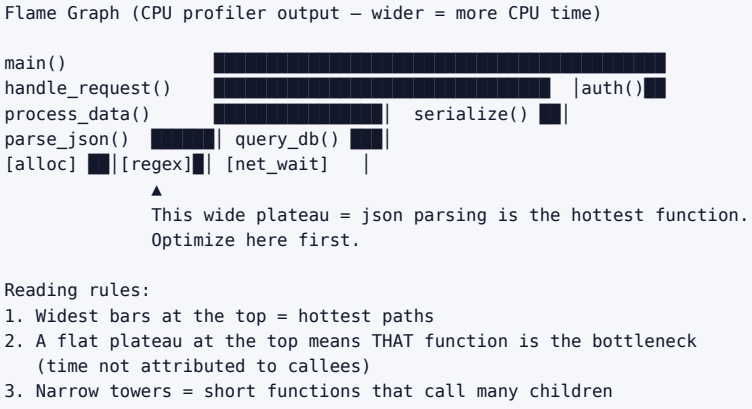
Interview takeaway: The fastest algorithmic optimization is often just making memory access patterns sequential (cache-friendly). An $O(n^2)$ algorithm with perfect cache behavior can beat O(n log n) with poor cache behavior for small N.

1.6 Architecture / Diagrams

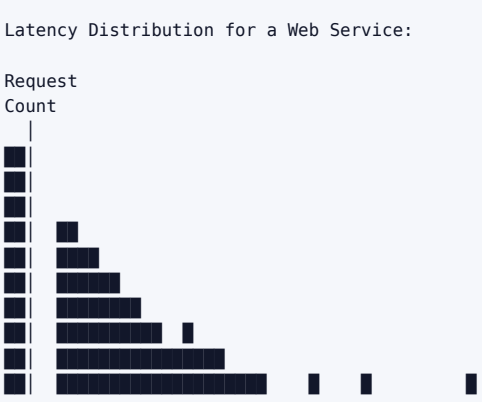
Memory Hierarchy with Latency Numbers



Flame Graph Sketch



Latency Percentile Distribution



```
──┼──────────────────────────────────────────────► Latency
  0   2   5  10  20  30  50  100 200 500   2000 ms

      |p50|     |p90|   |p99|  |p999|
      (10ms)  (30ms)  (100ms) (2000ms)

Note: p999 = 2000ms while p50 = 10ms — this "tail" matters
because at 1M requests/day, 1000 users hit 2-second latency.
```

Where Bottlenecks Hide in a Request Path

```
Client → DNS → CDN → Load Balancer → App Server → Cache → Database
  │        │     │         │              │           │        │
  │        │     │         │              │           │        └─ Slow query, missing index,
  │        │     │         │              │           │            lock contention, I/O
  │        │     │         │              │           └─ Cache miss → extra DB roundtrip
  │        │     │         │              └─ GC pause, CPU-bound code, thread exhaustion
  │        │     │         └─ Imbalanced routing → hot server
  │        │     └─ Cache miss → origin fetch
  │        └─ DNS lookup latency (should be <5ms)
  └─ Client-side rendering, JS bundle size

Checklist for each hop:
  □ Time from client perspective (tracing spans)
  □ Identify which hop is >50% of total latency
  □ Drill into that hop with profiling
```

1.7 The Performance Investigation Workflow

The **scientific method** applied to performance:

```
┌──────────────────────────────────────────────────┐
│                PERFORMANCE INVESTIGATION         │
│                                                  │
│  1. OBSERVE         Define the symptom precisely.│
│     "p99 latency increased from 50ms to 500ms    │
│      starting at 14:23 UTC on Tuesday"           │
│                          ↓                       │
│  2. MEASURE         Establish baseline with metrics.
│     CPU%, memory, I/O, network, thread count,    │
│     GC pauses, error rates, request rates        │
│                          ↓                       │
│  3. HYPOTHESIZE     Form a specific hypothesis.  │
│     "I think it's GC pauses because memory       │
│      usage also increased at 14:23"              │
│                          ↓                       │
│  4. PROFILE         Target the hypothesis.       │
│     GC logs, heap dump, allocation profiler      │
│                          ↓                       │
│  5. FIND BOTTLENECK  Confirm the single constraint.
│     "Old gen filling up, Full GC every 30s,      │
│      each pause = 800ms"                         │
│                          ↓                       │
│  6. FIX             Change one thing at a time.  │
│     Increase heap, fix the leak, upgrade to ZGC  │
│                          ↓                       │
│  7. RE-MEASURE      Confirm the fix works.       │
│     p99 back to 50ms, GC pauses < 10ms           │
│                          ↓                       │
│  8. DOCUMENT        Write up root cause + fix.   │
│     Post-mortem, runbook update                  │
└──────────────────────────────────────────────────┘
```

Key principle: Change one variable at a time. Otherwise you cannot know which change fixed the problem.

Premature optimization: Optimizing code before measuring where the actual bottleneck is. Donald Knuth: ”Premature optimization is the root of all evil.” **97% of the time**, performance problems are in the 3%

of code you would not intuitively suspect.

Load testing before and after:

- Use tools like `wrk`, `locust`, `k6`, `gatling`, `JMeter`
- Test at realistic concurrency (not just 1 thread)
- Test at percentiles, not just averages
- Run steady-state load test AND spike tests

1.8 Real-World Examples

Example 1: N+1 Query Problem (DB latency)

Scenario: A REST API that lists users and their orders. Naively coded:

```
>_ source
users = db.query("SELECT * FROM users")          # 1 query
for user in users:
    user.orders = db.query(                       # N queries (one per user)
        "SELECT * FROM orders WHERE user_id = ?",
        user.id
    )
```

For 1,000 users: **1,001 DB round-trips.** At 1ms each = 1 second latency.

Fix: JOIN or batch load:

```
>_ source
users = db.query("SELECT u.*, o.* FROM users u LEFT JOIN orders o ON u.id = o.user_id")
# 1 query, 1 round-trip, <10ms
```

Interview takeaway: N+1 is the most common performance bug in web apps. Always check SQL logs during profiling.

Example 2: Thread Pool Exhaustion (throughput collapse)

Scenario: Tomcat with default 200 threads. Upstream DB starts responding slowly (100ms → 500ms). Each thread now held 5× longer. Little's Law:

- Before: 200 threads / 100ms = 2,000 req/s
- After: 200 threads / 500ms = 400 req/s (80% drop!)
- Requests queue up, timeouts cascade, the service appears down.

Fix options:

1. Fix the DB (reduce latency) — correct fix
2. Add async I/O (threads not held during DB wait) — architectural fix
3. Increase thread pool (buys time but burns more memory) — palliative

Example 3: Cache Stampede at Black Friday

Scenario: E-commerce site. Product catalog is cached with 60-second TTL. At 00:00 Black Friday, millions of users hit the site simultaneously. Cache expired at 23:59. All requests miss cache, hit DB simultaneously. DB is overwhelmed and falls over.

Fix: Probabilistic early expiration (PER):

- Before TTL expires, randomly re-compute cache with probability proportional to remaining TTL
- No thundering herd because re-computation is spread over time

Or: **mutex lock on first miss** — one thread fetches from DB, others wait and then read from cache.

Example 4: False Sharing in Multithreaded Counter

Scenario: 4-thread counter updates:

```
struct Counters {
    long counter0;   // 8 bytes
    long counter1;   // 8 bytes  } same 64-byte cache line!
    long counter2;   // 8 bytes
    long counter3;   // 8 bytes
};
```

Threads 0-3 each update their own counter, but because all four share a cache line, every write invalidates the entire line on all other cores → **cache line ping-pong** → 4× slower than a single-threaded counter.

Fix: Pad to 64 bytes:

```
struct Counter {
    long value;
    char padding[56];  // pad to 64 bytes
};
```

1.9 Real-Life Analogies

One highway — every concept is a lane, a car, or a chokepoint in the same traffic system.

Concept	Analogy
Latency	How long one car takes to drive end-to-end from the on-ramp to the exit — door to door, including every red light and merge
Throughput	Cars per hour flowing through the highway system — the system's total productive output regardless of any one driver's trip time
Bandwidth	The number of lanes on the highway — more lanes means more cars can move in parallel, but a narrow merge point still caps you
Cache	An express lane with a pre-paid transponder: frequent commuters who already have clearance bypass the main toll plaza entirely and zip through
Cache miss	Your transponder isn't recognized — you're waved out of the express lane and sent back to the regular toll booth queue
Cache stampede	The express lane scanner breaks at rush hour; every commuter who relied on it floods the single open toll booth simultaneously, overwhelming it
Amdahl's Law	You add ten extra highway lanes — but every car still must pass through the single toll booth at the end. No matter how wide the road, that one booth is the ceiling on total throughput
Little's Law	The number of cars on the highway at any moment equals the arrival rate times the average trip duration — double the trip time with no extra on-ramps and the highway fills up, halving entry flow
GC pause	The highway authority closes all lanes for an emergency resurfacing crew. Every car stops. When lanes reopen, traffic is backed up for miles — a stop-the-world event
Flame graph	The traffic helicopter's heat map: the widest, reddest stretch of road is where cars spend the most time. That's where to send the work crews first
p99 latency	The unlucky 1% of drivers stuck behind a multi-car pile-up on the shoulder. The other 99% sail past; these few wait for the tow truck, and their delay has nothing to do with lane count
N+1 query	Instead of one truck delivering to all 1,000 houses on the route in a single pass, a driver returns to the depot after every single delivery to pick up the next parcel — 1,000 separate round-trips on the same road

Concept	Analogy
Connection pool	A fixed fleet of licensed taxis permanently stationed at the highway interchange. Passengers (requests) grab a waiting cab, ride, and the cab returns to the fleet — no cold-starting a new cab for every trip
False sharing	Two drivers in adjacent lanes both need to reach into the same glovebox mounted between the seats. Every time one grabs it, the other must wait, even though they're heading to completely different exits
Tail latency amplification	A convoy of 100 cars must all arrive before the toll gate opens. The gate stays shut until the last car clears its pile-up. One stalled car at the back holds everyone — the convoy's arrival time is the worst individual time, not the average
Bottleneck (Theory of Constraints)	A six-lane highway that narrows to a single-lane bridge in the middle. Repaving any other stretch or adding lanes anywhere else does nothing — the bridge is the constraint, and only widening it moves the needle
Profiling	The traffic helicopter hovering overhead, tracking where cars actually slow down with live camera feeds — instead of the city planner guessing which intersection is congested from his desk
Premature optimization	Resurfacing the fast, empty stretch of highway at mile 80 while the one-lane bridge at mile 40 is causing a five-mile backup. You spent the budget in exactly the wrong place
Memory hierarchy	Distance from the driver to the item: pocket (register, instant) → glove box (L1 cache, one second) → trunk (RAM, a short stop) → home garage (SSD, drive back home) → a warehouse in another city (network, a day's journey) — every extra stop costs time
Memory leak	Cars that enter the city's parking garages and never leave — their drivers walked away for good. Every hour, more spaces fill up permanently until the garages are full and new cars circle endlessly with nowhere to park
CPU-bound vs I/O-bound	CPU-bound: the highway is gridlocked because there are genuinely too many cars for the road capacity — you need more lanes. I/O-bound: cars are all idling at a railway crossing waiting for a slow freight train to pass — the road is empty but nothing moves until the crossing clears
Batching	A single bus carrying fifty passengers down the highway instead of fifty separate cars, each making the same trip one at a time — one toll payment, one lane slot, one fuel stop, fifty commuters delivered

1.10 Memory Tricks / Mnemonics

USE = ”Utilization, Saturation, Errors” → **”Uncle Sam's Equipment”** (Every resource must be checked for all three)

RED = ”Rate, Errors, Duration” → **”RED traffic light means stop and measure”** (Stop and look at rate, errors, how long it took)

Little's Law → **”L = λW”** → **”Little Lambda Waits”**

- L = items in system
- λ (lambda) = arrival rate
- W = wait time per item

Amdahl's Law → **”Parallel parts help, serial parts limit”**

- Remember: if 10% is serial → max 10× speedup regardless of CPU count (1/0.1 = 10)
- If 1% is serial → max 100× speedup
- Formula: 1/(serial fraction)

Latency numbers — ”0.5, 5, 50, 100, 100K, 500K, 10M” (nanoseconds)

- L1: 0.5 ns
- L2: 5 ns
- L3: 50 ns (approximately)
- RAM: 100 ns

- NVMe: 100,000 ns (100 µs)
- SSD: 500,000 ns (500 µs)
- HDD: 10,000,000 ns (10 ms)

p99 at scale → ”One in a hundred, but at Google scale that's millions”
Cache eviction → ”LRU = Least Recently Used → Least important to keep”
Profiling order → ”SAMI”

- **S**ample first (low overhead)
- **A**nalyze flame graph
- **M**easure the hot path
- **I**nstrument only what you need

GC pause generations → ”Young dies fast, Old lives long, Full GC costs most”
Bottleneck types → ”CPU, MEM, IO, NET, LOCK” → ”Can My I/O Never Lock?”

1.11 Common Interview Questions

Q1: "Your service got 10× slower overnight. How do you debug it?"

Model Answer:

Step 1 — **Define the symptom precisely.** ”When did it start? What exactly got slower — p50, p99, or throughput? Did anything change (deploy, traffic spike, config change)?”

Step 2 — **Check the four golden signals:**

- Latency: look at distributed trace spans (Jaeger/Zipkin) to find which service/query is slow
- Traffic: did request rate change?
- Errors: are there new errors?
- Saturation: any resource at 100%?

Step 3 — **Identify the layer:**

- Check CPU% on app servers
- Check DB slow query log
- Check cache hit rate
- Check GC logs for pause times
- Check thread pool utilization
- Check downstream service latencies

Step 4 — **Profile the bottleneck:** If CPU is high → flame graph If DB is slow → EXPLAIN ANALYZE on slow queries If GC → heap dump

Step 5 — **Fix one thing, re-measure, confirm improvement.**

Follow-up: ”What if the issue only happens at high load?” → Likely resource saturation (thread pool, connection pool, DB max connections). Test with load tester at realistic concurrency.

Follow-up: ”What if you can't reproduce it in staging?” → Use dark launch / shadow traffic, or add production instrumentation (low-overhead async sampling), or correlate with production metrics spikes.

Q2: "How would you design a caching layer for a social media feed?"

Model Answer:

Identify access patterns:

- Reads » Writes (feeds are read often)
- Data is user-specific (cannot share one cache entry for all)
- Some users are very popular (celebrities → hot keys)

Cache design:

- **L1:** In-process cache per app server (Guava Cache) — microsecond hits, limited size

- **L2:** Redis cluster — millisecond hits, gigabytes of capacity
- **DB:** PostgreSQL with read replicas

Cache key: `feed:{user_id}` → list of post IDs

Eviction: LRU with TTL (e.g., 5 minutes). Write-through on new post: invalidate feed cache for all followers (fan-out on write).

Hot key problem: Celebrity with 100M followers. Posting invalidates 100M cache keys simultaneously. Solution: asynchronous fan-out via message queue, or pull-on-read for large accounts.

Follow-up: "How do you handle cache invalidation?" → Event-driven: when a post is deleted, publish to Kafka, consumers invalidate relevant cache keys. Or: TTL-based eventual consistency (simpler, slightly stale).

Q3: "Explain Amdahl's Law and when it applies."

Model Answer:

Amdahl's Law states that the speedup from parallelizing a task is bounded by the serial portion. If P fraction can be parallelized: `Speedup = 1 / ((1-P) + P/N)`.

If 20% of code is serial (P=0.8): max speedup = 1/0.2 = **5**×, regardless of whether you use 10 or 10,000 CPUs.

Implication: Before adding more machines, identify and eliminate serial bottlenecks (global locks, sequential I/O, single-threaded coordinators).

Example: If your distributed system has a single-threaded coordinator that receives all task assignments, that coordinator is the serial fraction. Adding worker nodes beyond the coordinator's capacity provides no benefit.

Follow-up: "What is Gustafson's Law?" → Amdahl's Law assumes fixed problem size. Gustafson's Law observes that in practice, as we add more compute, we solve bigger problems. The serial fraction stays constant in absolute terms but shrinks as a fraction of total work. Relevant for HPC/ML training.

Q4: "What is tail latency and how do you reduce it?"

Model Answer:

Tail latency is the latency at high percentiles (p99, p99.9). It matters because:

1. At scale, high-percentile requests affect a significant number of real users
2. Fan-out amplification: a request calling 100 services will likely hit a slow one

Causes of tail latency:

- GC pauses
- Context switches to OS
- Resource contention (lock congestion, CPU scheduling delays)
- Occasional cold cache / page fault
- Network jitter
- Infrequent but slow code paths

Mitigation:

1. **Hedged requests:** Send to two replicas, use first response. ~5% extra load, dramatic p99 improvement.
2. **Timeouts with fallback:** Fail fast and return degraded response.
3. **Reduce GC pauses:** Use ZGC, reduce allocation rate, tune heap.
4. **Isolate slow paths:** Move slow operations async so they do not block fast paths.
5. **Load shedding:** Drop low-priority requests under load to protect latency for high-priority ones.

Q5: "How do you use profiling tools on a Java service in production?"

Model Answer:

Low-overhead options for production:

1. **async-profiler:** Sampling profiler using AsyncGetCallTrace. ~1-3% overhead. Can profile CPU, allocations, lock contention. Output: flame graphs.

```
./profiler.sh -d 30 -f /tmp/flame.html <pid>
```

2. **JFR (Java Flight Recorder):** Built into JVM since Java 11. Near-zero overhead. Captures CPU, GC, I/O, allocations, exceptions, lock contention.
3. **Heap dump on OOM:** `-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/tmp/`
4. **GC logging:** `-Xlog:gc*:file=/tmp/gc.log`

Never use instrumentation profilers in production — they add too much overhead and alter timing.
Follow-up: "How do you find a memory leak in Java?"

1. Monitor heap growth over time (JMX, Prometheus JVM metrics)
2. Take heap dump with `jcmd <pid> VM.native_memory` or `jmap -dump:live,...`
3. Analyze with Eclipse MAT — "Leak Suspects" report shows top memory consumers
4. Find the retention path (dominator tree view)
5. Fix: remove unnecessary references, use weak references for caches, implement eviction

1.12 Senior-Level Discussion Points

Tail Latency and Service Meshes

At FAANG scale, every downstream call must have:

- **Deadline propagation:** A request with a 200ms deadline propagates remaining time to each downstream service. If 150ms is already spent, downstream services get 50ms. They should not do work that cannot complete in time.
- **Context propagation:** Trace IDs, deadlines, and priority levels flow through headers (W3C Trace Context, gRPC deadlines).

Capacity Planning

Use Little's Law to calculate required concurrency:

```
Required servers = (Target throughput × SLO latency) / (1 server's target throughput × SLO latency)
```

Example: Target 100,000 req/s at p99 < 100ms. Each server can handle 500 req/s at <10ms.

- Need 100,000 / 500 = 200 servers minimum
- Add headroom: 200 × 1.5 = 300 servers (50% headroom is common)
- Plan for peak: if peak is 3× normal, need 600 servers

Cost optimization: Spot/preemptible instances (70% cheaper) for batch workloads. Reserved instances for steady-state. On-demand for peaks.

Trade-offs in Performance Work

Trade-off	Latency Cost	Reliability/Cost Benefit
Replication	Higher write latency (must wait for N replicas)	Higher availability, read scalability
Compression	CPU overhead per request	Lower bandwidth, reduced network costs
Batching	Higher individual latency (wait to batch)	Much higher throughput, lower per-item cost
Async processing	Response before work done	Higher throughput; harder error handling
Caching	Stale data risk	Much lower latency, reduced backend load

The Performance vs Reliability Trade-off

Fast path vs slow path:

- Synchronous, in-critical-path operations must be fast (cache reads, index lookups)
- Async, out-of-critical-path operations can be slower (metrics emission, async logging)

Circuit breakers: When a downstream service degrades, a circuit breaker trips and returns a fast fallback response, protecting the caller from cascading latency. Library: Hystrix (deprecated), Resilience4j.

Database Performance at Scale

- **Read replicas:** Scale reads, but replica lag introduces consistency window.
- **Read-your-own-writes:** After a write, route reads for that user to primary for 1-2 seconds.
- **Connection pooling:** PgBouncer (PostgreSQL), ProxySQL (MySQL). Critical — DB connections are expensive. Rule of thumb: DB can handle 100-300 connections; app servers need thousands; pool sits in between.
- **Query optimization hierarchy:** Index (cheapest) → query rewrite → schema change → partitioning → sharding (most expensive)

1.13 Typical Mistakes Candidates Make

1. **Optimizing without measuring.** ”I would add a cache” without knowing if the DB is even the bottleneck. Always state ”first I would measure to confirm.”
2. **Conflating latency and throughput.** Saying ”the system is slow” — slow in which dimension? Adding threads can increase throughput but increase per-request latency due to context switching.
3. **Ignoring tail latency.** Only discussing p50/average. FAANG cares about p99/p99.9 because at scale, a small percentage of slow requests is millions of users.
4. **Not considering cache invalidation.** Proposing caching without discussing staleness, invalidation, cold starts, and cache stampedes.
5. **Amdahl's Law misapplication.** Assuming adding more machines linearly increases throughput. Without identifying the serial bottleneck first, more machines may not help.
6. **Forgetting Little's Law implications.** When latency goes up, throughput falls proportionally unless you add concurrency. Candidates often miss that thread pool exhaustion compounds latency problems.
7. **Not mentioning measurement tools.** A senior engineer names specific tools: `perf`, `async-profiler`, `Jaeger`, `Prometheus + Grafana`, `EXPLAIN ANALYZE`. Vague answers signal inexperience.
8. **Single-machine thinking.** In distributed systems, bottlenecks can be at the network, load balancer, shared cache, or DB — not just the app server. Think end-to-end.
9. **Ignoring GC as a source of latency spikes.** For JVM services, GC pauses are often the root cause of p99 degradation. Always ask about the GC.
10. **Proposing premature optimization.** Suggesting micro-optimizations (bit manipulation, loop unrolling) before architectural issues (N+1 queries, missing indexes) are fixed.

1.14 How This Connects To Other Topics

Topic	Connection
Operating Systems	CPU scheduling affects context switch overhead. Virtual memory and page faults affect memory latency. `epoll`/`kqueue` enables async I/O. `mmap` eliminates user-kernel copy. NUMA affects memory latency for multi-socket systems.
Concurrency	Thread contention is a performance bottleneck. Lock-free data structures reduce contention. Goroutines/fibers allow millions of concurrent operations. False sharing is a concurrency + cache interaction.

Topic	Connection
Databases	Query planning, indexes, buffer pool (InnoDB buffer pool = L3 cache for DB), connection pooling, read replicas, write-ahead log (WAL) are all performance levers. EXPLAIN ANALYZE is a profiler for queries.
System Design	Horizontal vs vertical scaling decisions require capacity planning (Little's Law). Caching layer placement is architectural. Microservices fan-out creates tail latency amplification. CDN is a cache at the network edge.
Cloud / Observability	CloudWatch, Datadog, Prometheus+Grafana implement RED method. AWS X-Ray, Google Cloud Trace, Jaeger implement distributed tracing (latency per span). Auto-scaling is dynamic capacity adjustment.
Networking	TCP slow start affects connection reuse value. Nagle's algorithm causes latency in bursty small-packet workloads (disable with `TCP_NODELAY`). HTTP/2 multiplexing reduces head-of-line blocking. TLS handshake adds latency (use session resumption).
Algorithms & Data Structures	Cache-oblivious algorithms. B-trees are cache-friendly for disk (large nodes match disk pages). Hash maps have poor cache behavior due to random access. Sorted arrays enable binary search with good cache behavior.

1.15 FAANG Interview Tips

1. **Always measure first.** Open every performance answer with ”Before optimizing, I would instrument and measure to confirm the bottleneck.” This signals senior thinking.
2. **Name the method.** ”I would use the USE method to check CPU, memory, and I/O utilization/saturation/errors” shows you have a framework, not guesswork.
3. **Mention percentiles explicitly.** Say ”p99 latency” not ”latency.” Immediately signals you think about tail behavior.
4. **Invoke Little's Law when discussing concurrency.** ”If latency doubles, throughput halves unless we add proportional concurrency” is a perfect application.
5. **Know your latency numbers.** L1 cache = 0.5 ns, RAM = 100 ns, SSD = 100 µs, network LAN = 0.1 ms, network WAN = 50-150 ms. Interviewers will probe this.
6. **Tail latency at scale.** Mention the Google ”Tail at Scale” insight: p99 of one service becomes p50 at the system level when you fan out to many backends.
7. **Cache invalidation is hard.** When you propose a cache, immediately discuss consistency, invalidation, TTL, and stampede. Shows production experience.
8. **Connect to business impact.** ”Amazon found that 100 ms of additional latency costs ~1% in revenue” — this shows you understand why performance matters.
9. **Differentiate CPU-bound vs I/O-bound clearly.** Know the symptoms and diagnostics for each. Mixing them up is a red flag.
10. **Flame graphs.** If asked to read a flame graph or profiler output, always identify the widest bar at the top — that is where to focus first.

1.16 Revision Cheat Sheet

10-Minute Summary

Performance engineering = measure → profile → bottleneck → fix → verify. Never guess. The three key metrics are **latency** (time per request), **throughput** (requests per second), and **bandwidth** (channel capacity).

Tail latency matters because at scale, p99 of one service becomes p50 of a fan-out system. Track p99/p99.9 for user-facing services.

Key laws:

- Amdahl: speedup bounded by serial fraction → eliminate serial bottlenecks before adding hardware
- Little's: $L = \lambda W$ → if latency rises and concurrency stays fixed, throughput falls proportionally

Key methods:

- USE (per resource): Utilization, Saturation, Errors
- RED (per service): Rate, Errors, Duration

Bottleneck types: CPU (high CPU%), Memory (cache misses, swap), I/O (high iowait), Network (NIC saturation), Lock (many threads, low CPU).

Memory hierarchy: L1 (0.5 ns) → L2 (5 ns) → L3 (40 ns) → RAM (100 ns) → NVMe (100 µs) → HDD (10 ms). Cache locality is the biggest practical CPU optimization.

Caching: Cache-aside is default. TTL + active invalidation for consistency. Beware stampedes (use probabilistic early expiration or mutex on first miss).

GC: Leaks in GC languages = unintended references. Detect via heap dump + Eclipse MAT. Fix GC pauses by switching to ZGC or reducing allocation rate.

Profiling: Sampling profiler (low overhead, flame graphs) for CPU. Heap dump + allocation profiler for memory. Always profile in a representative environment.

Key Points

- **Measure before optimizing** — 97% of performance problems are where you do not expect
- **p99 > p50** — tail latency affects real users and amplifies in distributed systems
- **Little's Law** — if latency goes up, throughput goes down proportionally
- **Amdahl's Law** — serial fraction caps the benefit of parallelization
- **USE for infra, RED for services** — complementary methods covering all four golden signals
- **Cache invalidation is the hardest part** — always address consistency when proposing a cache
- **N+1 queries** — the most common, most impactful DB performance bug
- **False sharing** — invisible multi-thread performance killer; pad to cache line size
- **Flame graphs** — widest bar at top = where to focus first
- **Latency numbers** — know them by heart; they appear in system design interviews constantly

Cheat Sheet Table

Concept	Formula / Rule	Key Number
Little's Law	L = λW (throughput = concurrency / latency)	—
Amdahl's Law	S = 1/((1-P) + P/N)	5% serial → max 20× speedup
p99 fan-out	P(any slow) = 1-(1-p)^N	N=100, p=1% → 63% affected
L1 cache hit	~0.5 ns	1 ns ≈ 1 CPU cycle
RAM access	~100 ns	200× slower than L1
NVMe SSD	~100 µs	1000× slower than RAM
Network LAN	~100 µs RTT	Same as NVMe
Network WAN	~50-150 ms	500,000× slower than L1
Cache line	64 bytes	Pad structs to avoid false sharing
USE method	Utilization, Saturation, Errors	Per resource
RED method	Rate, Errors, Duration	Per service
GC strategy	ZGC < 1ms pauses	Java 15+, low-latency prod
Hedged requests	~5% extra load	Cuts tail latency dramatically
Connection pool	Reuse TCP connections	DB: ~100-300 max connections

Most Important Concepts

1. **Measure first, always** — profiling > guessing
2. **Little's Law** — throughput = concurrency / latency
3. **Amdahl's Law** — serial fraction is the ceiling

4. **Tail latency** — p99 at scale affects millions of users
5. **USE + RED** — the complete observability framework
6. **Memory hierarchy** — cache locality is the biggest CPU win
7. **Cache invalidation** — the hardest problem in distributed systems
8. **Flame graphs** — visual profiling, widest bar at top
9. **GC pressure** — leading cause of JVM latency spikes
10. **Bottleneck isolation** — CPU vs memory vs I/O vs network vs lock

Last updated: 2026-06-14 | Topic: Performance Engineering | Level: FAANG Senior

2 Visual Reference

A set of figures distilling the key quantitative intuitions of this topic.

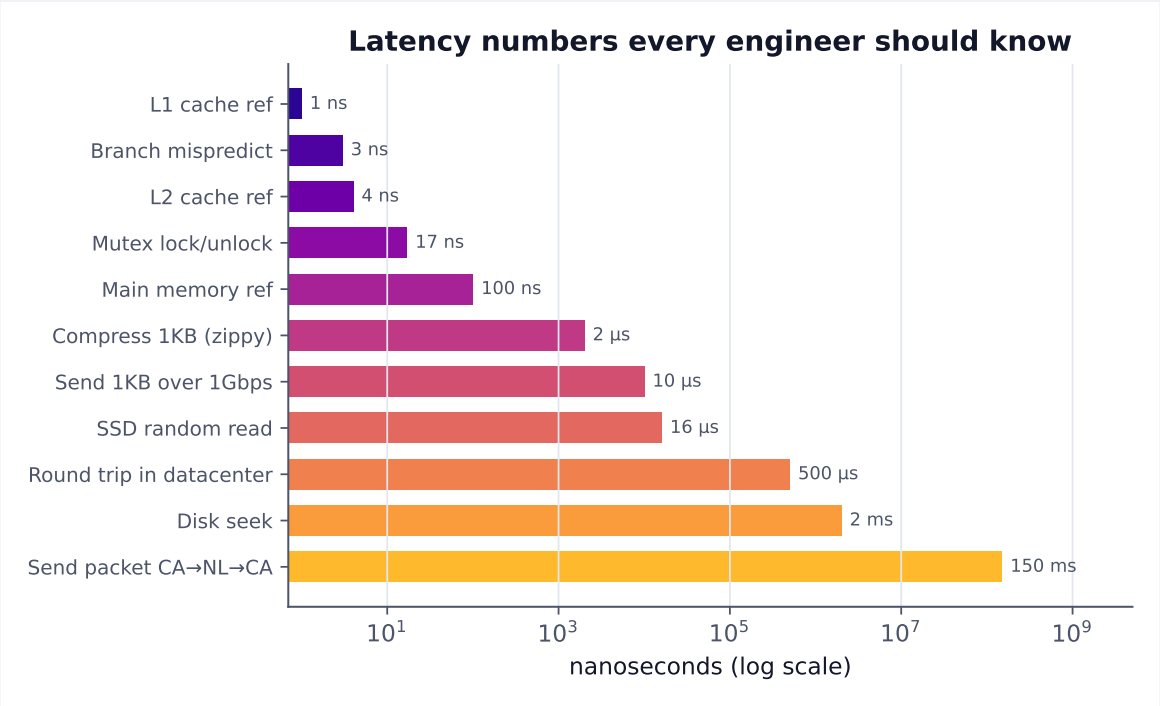


Figure 1. Orders of magnitude separate cache, memory, SSD, disk and network. Designing for performance means counting which tier each operation touches.